Merchant Intelligence Platform Architecture

End-to-End System Design for AI-Powered Merchant Credit Scoring

Moniepoint · Engineering Division · Confidential

Document Type	System Architecture & Design Patterns
Version	1.0 — Initial Release
Stack	FastAPI · PostgreSQL · Redis · Claude AI · React
Author	Engineering Team
Date	15 May 2026

TABLE OF CONTENTS

What's Inside

SECTION 01

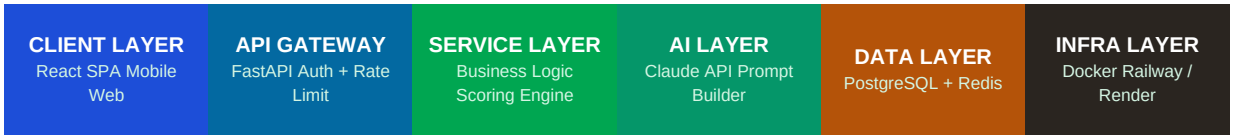
System Overview

The Moniepoint Merchant Intelligence Platform is a full-stack, AI-augmented credit scoring system designed to assess the creditworthiness of Nigerian small businesses using behavioral transaction data rather than traditional credit bureau scores. The architecture follows a layered, service-oriented design with clear separation of concerns across six functional zones.

Core Design Principles

Principle	Description	Applied To
Separation of Concerns	Each layer has a single responsibility. Routes validate. Services compute. Repositories store.	All layers
Deterministic Scoring	Credit scores are computed from pure math — never by AI. Claude explains; it does not calculate.	Scoring Engine
Async-First	All I/O — DB, cache, and AI calls — is non-blocking. The server never waits.	FastAPI + Async SQLAlchemy
Streaming by Default	AI analysis streams token-by-token to the UI. Users see output immediately, not after 8s wait.	Claude + SSE
Cache-Heavy	Score results and AI analyses are cached in Redis. AI is only called when data materially changes.	Redis layer
Zero Trust internally	Every service authenticates. No service trusts another implicitly. JWT + API keys at every boundary.	Security layer

High-Level Architecture Zones



Data flows left to right: client requests enter through the API Gateway, are processed by service layer logic, enriched by the AI layer when needed, and persisted or retrieved from the data layer.

SECTION 02

Application Layers

Project Directory Structure

The codebase is organized into five top-level packages. Each package owns exactly one concern. No cross-package imports except through explicitly defined interfaces.

Path	Owns	Pattern
api/v1/	HTTP interface — routes, request parsing, response shaping	Thin controllers — no business logic
services/	All business rules — scoring, AI orchestration, caching decisions	Service classes with dependency injection
repositories/	All database access — queries, inserts, updates	Repository pattern — zero raw SQL in services
models/	SQLAlchemy ORM models — DB schema as Python classes	Declarative base — async-compatible
schemas/	Pydantic v2 models — request/response validation and serialization	Strict mode — no extra fields allowed
core/	Config, DB connection, security utils, middleware	Singleton pattern for shared resources
workers/	Background jobs — score recomputation, data ingestion tasks	Celery tasks with Redis broker
tests/	Unit + integration tests — one test file per module	Pytest + AsyncIO + factory_boy fixtures

The Dependency Rule

■ Strict Layer Dependency Rule

Routes may only import from: services, schemas

Services may only import from: repositories, schemas, core

Repositories may only import from: models, core

No layer may import from a layer above it — ever

AI service is a service-layer concern — routes never call Claude directly

SECTION 03

Database Architecture

PostgreSQL — Primary Data Store

PostgreSQL is the source of truth for all merchant data, transactions, scores, and analyses. Chosen over NoSQL because merchant credit scoring requires relational integrity — a transaction must belong to exactly one merchant, scores must reference verified transaction windows.

Core Schema Tables

Table	Key Columns	Purpose	Indexes
merchants	id, business_name, type, city, state, moniepoint_id, onboarded_at, is_active	Master merchant record	moniepoint_id (unique), city+type (composite)
transactions	id, merchant_id (FK), amount, txn_type, category, occurred_at, pos_terminal_id	Raw POS transaction ledger	merchant_id+occurred_at (composite), occurred_at (BRIN for time-range queries)
credit_scores	id, merchant_id (FK), total_score, grade, computed_at, signal_json (JSONB), model_version	Computed score snapshots — never mutated	merchant_id+computed_at, grade
signal_details	id, score_id (FK), signal_name, raw_value, normalized_score, weight	Individual signal breakdown per score	score_id
ai_analyses	id, merchant_id (FK), score_id (FK), content_text, prompt_hash, model, tokens_used, created_at	Cached Claude-generated narratives	merchant_id+created_at, prompt_hash (for dedup)
recommendations	id, merchant_id (FK), score_id (FK), title, description, expected_impact, priority, category	AI-generated improvement actions	merchant_id, priority
score_history	id, merchant_id (FK), score, grade, recorded_at	Lightweight score trend table — append only	merchant_id+recorded_at

Key Database Design Decisions

JSONB for signal data

Signal weights and parameters evolve as the model improves. Storing them as JSONB in credit_scores preserves the exact computation parameters used for that snapshot — you can always audit why a merchant got a specific score at a specific time.

BRIN Index on transactions.occurred_at

Transactions are an append-only log that grows very large. BRIN (Block Range Index) is orders of magnitude smaller than a B-tree index and performs excellently for date-range queries on insert-ordered tables.

Scores are immutable snapshots

credit_scores rows are never updated — a new row is inserted each time a score is recomputed. This gives you a full audit trail for regulatory compliance and lets you track score drift over time.

Connection Pooling via asyncpg

Using SQLAlchemy async engine with asyncpg driver. Pool size: min=5, max=20. Under high load, requests queue rather than fail — critical for a fintech product where dropped requests erode trust.

Redis — Caching & Queue Layer

Key Pattern	Value	TTL	Purpose
score:{merchant_id}	JSON score object	1 hour	Latest score — avoids recompute on every request
analysis:{merchant_id}:{score_hash}	Analysis text string	7 days	Cached Claude output — only expire when score changes significantly
ratelimit:{ip}:{endpoint}	Request count integer	60 seconds	Per-IP rate limiting on AI endpoints
session:{token_hash}	User session JSON	24 hours	JWT session cache for fast auth lookup
txn_ingest_queue	List of merchant IDs pending recompute	No TTL	Background worker queue — triggers score recompute on new txn batch
leaderboard:score	Sorted set of merchant_id:score	6 hours	Portfolio-level score distribution for ops dashboard

SECTION 04

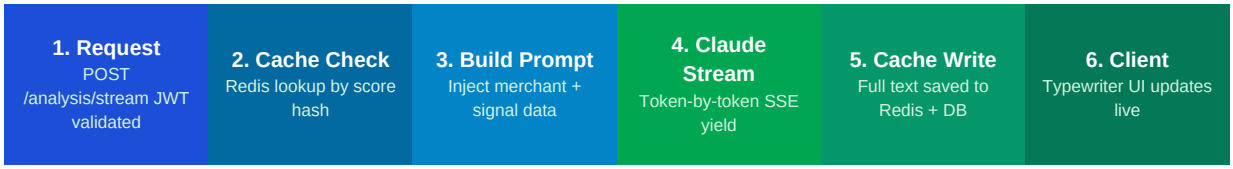
API Design & Endpoints

All endpoints are versioned under /api/v1/. Authentication uses JWT Bearer tokens issued at login. The AI analysis endpoint uses Server-Sent Events (SSE) for real-time streaming. All responses follow a consistent envelope: {data, meta, error}.

Endpoint Reference

Method	Path	Auth	Description
POST	/auth/token	None	Issue JWT from API key or credentials
GET	/merchants	JWT	Paginated merchant list with latest scores
POST	/merchants	JWT + Admin	Onboard a new merchant
GET	/merchants/{id}	JWT	Full merchant profile
PATCH	/merchants/{id}	JWT + Admin	Update merchant details
GET	/merchants/{id}/transactions	JWT	Paginated txn history — filter by date range
POST	/merchants/{id}/transactions/bulk	API Key	Ingest batch transactions from POS system
GET	/merchants/{id}/score	JWT	Current score + all 6 signal breakdowns
POST	/merchants/{id}/score/recompute	JWT	Force score recalculation — queues background job
GET	/merchants/{id}/score/history	JWT	Score trend over time (sparkline data)
POST	/merchants/{id}/analysis/stream	JWT	SSE stream — real-time Claude analysis
GET	/merchants/{id}/analysis/latest	JWT	Most recent cached analysis
GET	/merchants/{id}/recommendations	JWT	Improvement roadmap for this merchant
GET	/portfolio/stats	JWT + Admin	Aggregate portfolio health (score distribution, risk buckets)
GET	/health	None	Service health check for load balancer

Streaming Analysis Endpoint — How It Works



Why SSE over WebSockets for streaming?

SSE (Server-Sent Events) is unidirectional and stateless — the server pushes, the client listens. This is all we need for AI streaming and is far simpler to implement, scale, and proxy than WebSockets.

FastAPI yields SSE chunks with: yield f'data: {chunk}\n\n' inside an async generator, wrapped in StreamingResponse(content, media_type='text/event-stream').

Nginx and all cloud load balancers handle SSE natively with no special config beyond setting proxy_buffering off.

SECTION 05

AI Architecture

The Golden Rule: Claude Explains. Python Scores.

The credit score is always computed deterministically in Python from raw transaction data. Claude's role is purely narrative — it receives already-computed numbers and generates a plain-language analysis. This keeps the scoring model auditable, consistent, and regulatorily defensible. If Claude ever became unavailable, scores would still compute correctly.

Scoring Engine — The 6 Signals

Signal	Computation Method	Weight	Data Source
Revenue Consistency	Coefficient of variation (std/mean) on monthly revenue. Inverted and scaled 0-100.	25%	transactions (last 6mo)
Transaction Frequency	Avg daily txn count vs peer group median. Capped at 100.	20%	transactions (last 90d)
Business Tenure	Months since first Moniepoint transaction. Plateau at 36 months = 100.	15%	merchants.onboarded_at
Cash Flow Coverage	Avg monthly revenue / estimated monthly repayment (loan amount / 12). Capped at 100.	20%	transactions + loan params
Seasonal Resilience	Min monthly revenue / max monthly revenue over 12 months. 1.0 = no seasonality.	10%	transactions (last 12mo)
Weekend Revenue Mix	Weekend revenue / total revenue. Optimal band is 30-60%. Penalties outside band.	10%	transactions (last 90d)

Prompt Architecture — Three-Layer System

Prompts are not ad-hoc strings. They are structured, versioned templates with three layers that separate stable instructions from dynamic data.

Layer	Content	Updates When
System Prompt	Claude's persona, tone, constraints, output format rules, Nigerian context instructions. Stored in core/prompts.py as a named constant.	Model version changes or business rules change
Context Injection	Merchant name, type, location, tenure. Score, grade, trend direction. All 6 signal values + descriptions. Formatted into a structured data block.	Every API call — merchant-specific

Layer	Content	Updates When
Task Instruction	The specific output requested: credit report, recommendations, or risk summary. Word limit, audience (loan officer vs merchant), formatting constraints.	Different endpoint types

AI Caching Strategy — When to Call Claude

Condition	Action	Rationale
Analysis exists in Redis for this merchant+score_hash	Return cache immediately — no API call	Identical inputs produce identical useful outputs
Score changed by < 5 points since last analysis	Return cached analysis — no API call	Small score movements don't change the narrative materially
Score changed by >= 5 points	Invalidate cache, call Claude, store result	Meaningful change warrants fresh analysis
Cache miss but DB has recent analysis (< 7 days)	Warm Redis from DB, return cached	Redis eviction doesn't cause unnecessary AI calls
Merchant explicitly requests refresh	Bypass cache, call Claude, overwrite cache	User agency respected — but rate-limited to 3x/day
AI call fails (timeout or API error)	Return last known analysis with staleness warning	Graceful degradation — never block the UI

SECTION 06

Frontend Architecture

The frontend is a React SPA using Vite as the build tool. State management uses Zustand — lightweight, no boilerplate, and sufficient for this application's complexity. The UI has three primary concerns: data display, real-time streaming, and score visualization.

Directory Structure

Path	Contains	Pattern
src/api/	All fetch calls — one file per backend resource (merchants.js, scoring.js, analysis.js)	Service module — no fetch() calls in components
src/components/	Reusable UI atoms: ScoreRing, SignalBar, MetricCard, StreamingText	Presentational only — no data fetching
src/features/	Feature modules: MerchantList, CreditReport, Portfolio. Each owns its route, container, and local state.	Feature-first — collocated concerns
src/hooks/	Custom hooks: useSSEStream, useMerchantScore, useScoreHistory	Encapsulate all side effects and async logic
src/store/	Zustand stores: merchantStore, uiStore, authStore	Global state — minimal, flat, serializable
src/utils/	Score formatters, currency formatters, date helpers	Pure functions — no side effects

SSE Streaming Hook — useSSEStream

This custom hook manages the real-time AI text stream from the backend. It handles connection, token accumulation, error recovery, and cleanup.

```
// src/hooks/useSSEStream.js
export function useSSEStream(merchantId) {
const [text, setText] = useState('');
const [status, setStatus] = useState('idle'); // idle | streaming | done | error
const esRef = useRef(null);
const startStream = useCallback(async () => {
setText(''); setStatus('streaming');
const token = getAuthToken();
esRef.current = new EventSource(
`/api/v1/merchants/${merchantId}/analysis/stream`,
{ headers: { Authorization: `Bearer ${token}` } }
);
esRef.current.onmessage = (e) => {
```

```
  if (e.data === '[DONE]') { setStatus('done'); esRef.current.close(); return; }
  setText(prev => prev + e.data); // append each token
  };
  esRef.current.onerror = () => setStatus('error');
  }, [merchantId]);
  useEffect(() => () => esRef.current?.close(), []); // cleanup on unmount
  return { text, status, startStream };
  }
```

SECTION 07

Infrastructure & Deployment

Container Architecture — Docker Compose (Dev)

Service	Image	Port	Role
api	python:3.12-slim (custom)	8000	FastAPI application server — Uvicorn with 4 workers
worker	same as api	—	Celery worker for background score recomputation
postgres	postgres:16-alpine	5432	Primary database — data persisted on named volume
redis	redis:7-alpine	6379	Cache + Celery broker — append-only log persistence
frontend	node:20-alpine (dev)	5173	Vite dev server — hot module replacement
nginx	nginx:alpine	80/443	Reverse proxy — routes /api to FastAPI, / to frontend

Production Deployment Strategy

Backend (Railway or Render)

FastAPI runs as a Docker container on Railway. Railway auto-detects the Dockerfile and deploys on every push to main. Zero-downtime deployments via rolling restarts. Environment variables injected at runtime — never in the image.

Database (Supabase or Railway Postgres)

Managed PostgreSQL with daily automated backups. Connection string injected via DATABASE_URL env var. Alembic handles schema migrations — run as a pre-deploy step.

Frontend (Vercel)

React app deployed to Vercel as a static build. Vercel's edge network serves assets from the CDN closest to the user. API calls proxy through /api rewrites to avoid CORS complexity.

Redis (Upstash)

Upstash provides serverless Redis with per-request billing — cost-effective for a demo or early-stage product. TLS enforced. Connection via REDIS_URL env var.

Environment Strategy

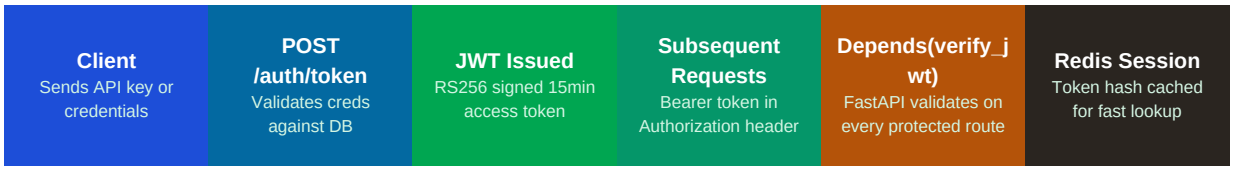
Environment	Purpose	Database	AI Calls	Branch
local	Development — full hot reload, debug logging	Docker Postgres	Real (dev key)	feature/*
staging	Pre-production — mirrors prod config exactly	Managed Postgres (staging DB)	Real (staging key)	develop

Environment	Purpose	Database	AI Calls	Branch
production	Live traffic — optimized, monitored, alerts on	Managed Postgres (prod DB)	Real (prod key)	main

SECTION 08

Security Architecture

Authentication Flow



Security Controls

Control	Implementation	Protects Against
JWT Authentication	RS256-signed tokens, 15-min access + 7-day refresh. Issued by /auth/token.	Unauthorized API access
API Key for POS Ingest	Separate high-entropy API key for the /transactions/bulk endpoint used by POS terminals.	Unauthorized transaction injection
Rate Limiting	Redis-backed per-IP limits. AI endpoints: 10 req/min. Score endpoints: 60 req/min.	Abuse, runaway AI costs
Environment Secrets	All secrets (DB URL, ANTHROPIC_API_KEY, JWT private key) in env vars. Never in code or Docker images.	Secret leakage
SQL Injection	SQLAlchemy ORM with parameterized queries exclusively. No raw SQL strings.	SQL injection
CORS	Origin whitelist in FastAPI CORSMiddleware — only frontend domain allowed in production.	Cross-origin attacks
HTTPS Everywhere	TLS terminated at Nginx/load balancer. HTTP → HTTPS redirect enforced. HSTS headers set.	Man-in-the-middle
Input Validation	Pydantic v2 strict mode on all request schemas. Max field lengths enforced.	Malformed input, oversized payloads
Prompt Injection Guard	Merchant data fields are sanitized and injected into prompts as structured data blocks, not raw user strings.	Prompt injection via merchant data

SECTION 09

Technology Stack Summary

Complete Dependency Reference

Layer	Technology	Version	Why This Choice
Backend Framework	FastAPI	0.115+	Async-native, automatic OpenAPI docs, best-in-class Pydantic integration
ASGI Server	Uvicorn	0.30+	Production-grade ASGI server, used with Gunicorn for multi-worker prod deploys
ORM	SQLAlchemy	2.0 (async)	Industry standard, async support in v2, excellent migration tooling via Alembic
DB Driver	asyncpg	0.29+	Fastest async PostgreSQL driver available for Python — 3x faster than psycopg2
Migrations	Alembic	1.13+	SQLAlchemy's native migration tool — auto-generates migrations from model changes
Validation	Pydantic	v2	Rust-compiled core — 5-17x faster than v1. Strict mode. First-class FastAPI support.
Caching / Queue	Redis (via redis-py)	5.0+	Industry standard for caching and lightweight task queuing
Background Jobs	Celery	5.3+	Mature, battle-tested async task queue — handles score recomputation jobs
AI SDK	anthropic (Python)	0.30+	Official Anthropic SDK with native async streaming support
Auth / JWT	python-jose + passlib	Latest	JWT signing and password hashing — standard FastAPI auth pattern
Testing	pytest + pytest-asyncio	Latest	Async test support, factory_boy for fixtures, httpx for async API testing
Primary Database	PostgreSQL	16	ACID compliance, JSONB support, excellent for relational + semi-structured data
Cache / Broker	Redis	7	Sub-millisecond reads, pub/sub for SSE fanout, reliable Celery broker
UI Framework	React	18	Component model, hooks for streaming state, massive ecosystem

Layer	Technology	Version	Why This Choice
Build Tool	Vite	5+	10-100x faster than webpack/CRA for dev. Native ESM. Excellent DX.
State Management	Zustand	4+	Minimal boilerplate. No Redux complexity needed at this scale.
HTTP Client	Axios + TanStack Query	Latest	TanStack Query handles caching, refetch, loading states automatically
Charts	Recharts	Latest	React-native charting — composable, lightweight, no canvas complexity
Styling	Tailwind CSS	3+	Utility-first — fast iteration, consistent design system, no CSS file sprawl
Containerization	Docker + Compose	Latest	Reproducible environments. Compose for local dev, Docker for prod deploys.
Backend Hosting	Railway	—	Git-push deploys, managed Postgres add-on, free tier sufficient for demos
Frontend Hosting	Vercel	—	CDN-first, instant global deploys, native Vite support
Managed Redis	Upstash	—	Serverless Redis — pay per request, TLS, free tier for demos
CI/CD	GitHub Actions	—	Run tests + lint on PR, auto-deploy on merge to main

★ For the Pitch Demo — Simplified Stack

You do not need Celery, Alembic, or multi-worker Uvicorn for a demo. Start with:

Backend: FastAPI + SQLite (via aiosqlite) + Redis (optional) — zero infrastructure setup

AI: Direct anthropic SDK calls, synchronous is fine for a demo

Frontend: The HTML/JS demo file already built — or Vite + React if you want to show the full stack

Deploy: Railway free tier for backend, Vercel for frontend — live URL in 30 minutes

Impress them with architecture knowledge in conversation, not by over-engineering the demo itself

End of Document · Moniepoint Merchant Intelligence Platform · Architecture v1.0 · Confidential